當足球遇上程式碼—我的 FBref 球員數據分析專題

申請人：葉陽甫　|　執行時間：113學年度下學期(高一下)　|　個人自主學習專題

身為球員，也是球迷，足球也成為我第一次把程式設計用來解決自己感興趣問題的題目。

專題名稱	FBref 球員數據分析專題
執行時間	113學年度下學期(高一下)
專題性質	個人自主學習專題
使用技術	Python、pandas、xlwings、網頁爬蟲

動機與起點

身為足球迷，我常和朋友爭論「哪位球員近期狀態較好」，但主觀感覺往往無法說服彼此。於是我想到：與其憑感覺爭論，不如用數據客觀比較。結合對程式設計的興趣，我設計足球數據爬蟲與視覺化儀表板，完成這個 FBref 球員數據分析專題。

資料來源：**FBref**（https://fbref.com/en/）
全球知名足球數據庫，提供球員、球隊與比賽的公開統計資料。

本專題以 **Python** 網頁爬蟲自動擷取，
作為球員數據分析與視覺化的基礎資料。

01 輸入球員

02 擷取 FBref 資料

03 整理兩組數據

04 寫入 Excel

05 視覺化比較

技術實作

以 Python 網路爬蟲擷取足球數據網站 FBref 的兩組資料：「近五場比賽表現」與「生涯重要統計」，再用 **pandas 與 xlwings 整理進 Excel**，透過資料比較分析球員的近期狀態與場上定位，把憑感覺的爭論轉化為可以攤開討論的證據。

▶ 專題報告書

挑戰與轉折

01 效率優化的代價：專案初期成功抓取單一球員數據後，我想效率更高，把程式改成能一次串接多名球員、短時間內連續發出大量請求。

02 意外的封鎖：程式在短時間內連續發出大量請求，不到一分鐘便觸發網站的反爬蟲 / 流量限制機制，IP 位址隨即無法正常存取。

03 治標的嘗試：我先用 VPN 更換 IP 位置，程式順利跑完，但隔天發現新的 IP 同樣被封鎖。

04 真正的問題：這讓我開始理解，問題不在於如何繞過封鎖，而是自己的請求方式對伺服器造成了過度負擔。

反思與佐證

從失敗中修正：理解爬蟲禮儀

我因此回頭研究「**爬蟲的禮儀**」，理解到兩個關鍵原則：網站可能透過 rate limiting 限制短時間內過多的請求，而較合宜的爬蟲應在每次請求之間加入適當間隔，降低伺服器負擔。這段經歷讓我寫下：

> **「這次失敗讓我發現，能把程式跑起來還不夠，也必須先理解網站的運作限制與使用規範。」**

足球經驗提供了問題意識，資料分析與程式設計則讓我把主觀觀察轉化為可比較的證據；真正的程式能力，也包含理解限制、尊重規範，並在失敗後找出可持續的解法。

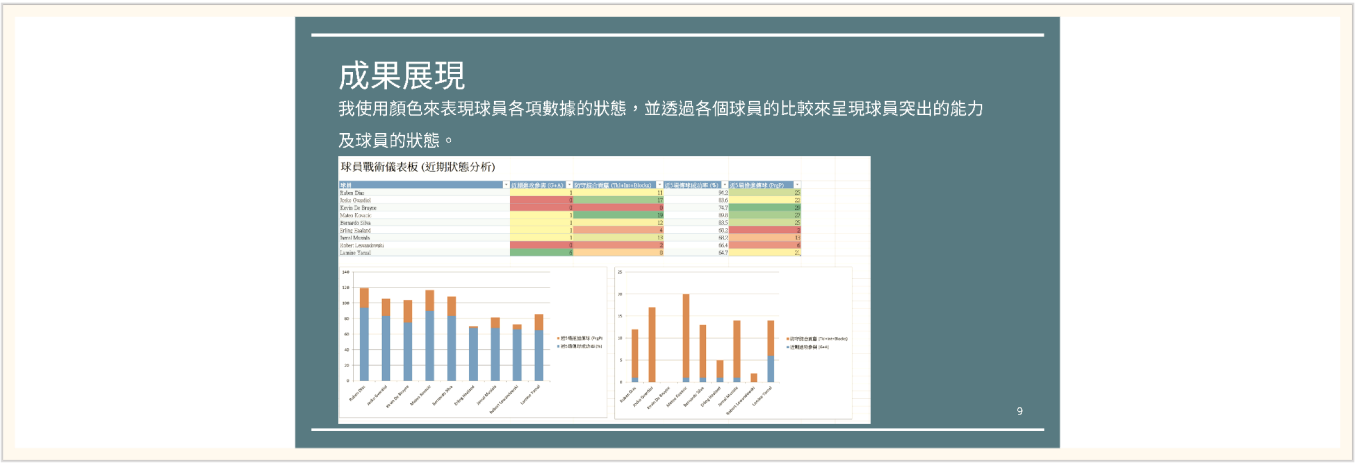
核心程式片段

▶ 完整程式碼與專案檔案

```
for player in players:
    df = scrape_player_stats(player)    # 一次串接多名球員，批次查詢
    time.sleep(3)                       # 加入請求間隔，避免觸發伺服器 rate limiting
    save_to_excel(df, player)
```

成果呈現

▶ 球員數據試算表



▲ 以色階與圖表呈現球員近期狀態，讓表現差異與場上定位更容易判讀

A | 爬取資料的清理與整合

B | 以四象限圖輔助戰術定位判讀